IMU-BASED VEHICLE DYNAMICS MONITORING SYSTEM

Technical Report

Design, Testing and Data Interpretation of a Low-Cost Six-Axis Motion Logger on a Scale Vehicle.

Prepared By

Krishang Srivastava

IMU-Based Vehicle Dynamics Monitoring Project

November 2026

Executive Summary

Background:

Every time a car accelerates, brakes, turns or rides over a bump, it moves in six ways at once: it speeds up or slows down along three axes and rotates about the same three. Engineers call these the longitudinal, lateral and vertical accelerations and the roll, pitch and yaw rates. Together they describe almost everything about how a vehicle is behaving, which is why stability control, airbag and rollover systems in modern cars are built around sensors that measure them.

Professional data-acquisition systems that record this motion are expensive and are not easy to get hold of as a student. At the same time, the sensor inside many of them, a MEMS inertial measurement unit (IMU), is now available as a small module that costs less than a meal out. That raised a simple question for us: how much real vehicle behaviour can a low-cost IMU actually capture if it is mounted properly and the data is read carefully?

Aim:

The aim of this project was to build a compact, self-contained logger that records all six motion channels during a run, and then to turn the recorded numbers into an honest reading of what the vehicle did. We wanted to go beyond "the sensor works" and see whether turns, stops, vibration and impacts could each be identified, and where the limits of a cheap sensor start to show.

Methodology:

The logger uses an MPU6050 six-axis IMU read by an ESP32 microcontroller over I²C at about 100 samples per second. Each sample is time-stamped and written to a microSD card as one row of a CSV file. The electronics sit in a three-part 3D-printed holder fixed to the roof of a model car and run from a power bank, so nothing is tethered to the ground during the test.

A 17.8 s run was recorded and imported into MATLAB, where each channel was plotted against time. The first rows of the file, taken with the car at rest, were used to check the sensor offsets and orientation before the moving data was interpreted channel by channel and then across channels.

Results:

At rest the sensor read a total of 0.996 g with very little noise, which confirmed it was working and close to level. During the run the **yaw rate** gave the cleanest picture: a series of distinct turns at 100 – 200 °/s, each matched by a sustained lateral acceleration of 0.3 – 0.5 g. Two stops were clearly visible as flat lines on every channel. The **vertical channel** was very busy, swinging between about −0.36 g and 1.93 g, because the small car has almost no suspension isolation.

Three moments, at about 11.9 s, 12.6 s and 15.4 s, showed sharp spikes on all six channels at once, which is the signature of an impact rather than a manoeuvre. These spikes also flattened at about ± 2 g, showing that the accelerometer had reached the end of its range. Comparing the signs of the yaw and lateral signals revealed something we had not expected: the sensor board appears to have been mounted turned through 180°.

Conclusions:

A logger built from hobby-grade parts was able to record and separate turns, stops, vibration and impacts on a moving vehicle. Just as useful were the things the data told us about the logger itself: the range setting was too small for impacts, and orientation has to be checked before every run, not

assumed. Those lessons, along with calibration and filtering, form the next steps in developing the system into a tool that can be trusted on a full-size vehicle.

TABLE OF CONTENTS

1. Introduction

2. Project Objectives

3. System Design

- 3.1 System Architecture
- 3.2 Hardware Components
- 3.3 Wiring
- 3.4 3D-Printed Sensor Holder
- 3.5 Final Prototype on the RC Car

4. Data Acquisition and Logging

- 4.1 Sampling and File Format
- 4.2 Sample Raw Data
- 4.3 Available Data Channels
- 4.4 Static Baseline Check

5. Acceleration Analysis

- 5.1 Longitudinal Acceleration
- 5.2 Lateral Acceleration
- 5.3 Vertical Acceleration

6. Angular Velocity Analysis

- 6.1 Roll Rate
- 6.2 Pitch Rate
- 6.3 Yaw Rate

7. Reading the Channels Together

- 7.1 Turns: Yaw Rate and Lateral Acceleration
- 7.2 Impact Events
- 7.3 Sensor Range Check
- 7.4 Axis Orientation Check

8. Automotive Applications

9. Limitations and Future Work

10. Conclusions

11. References

Appendix A – MATLAB Script

1. Introduction

If you sit in the back seat of a car with your eyes closed, you can still tell a surprising amount about what the driver is doing. You feel yourself pushed back when the car accelerates, thrown forward when it brakes, pressed sideways in a corner and bounced up and down over a rough road. Your body is acting as a crude motion sensor. An **inertial measurement unit** does the same job, only it puts a number on each of those sensations many times a second.

Vehicle engineers describe this motion using a set of axes fixed to the car (Figure 1.1). The X-axis points forward, the Y-axis to the side and the Z-axis upward. Movement along these axes gives the **longitudinal, lateral and vertical accelerations**, and rotation about them gives the **roll, pitch and yaw rates**. These six quantities are the basic language of vehicle dynamics. Yaw rate and lateral acceleration tell you how a car is turning, longitudinal acceleration tells you how hard it is accelerating or braking, and the vertical and rotational channels describe ride and body motion.

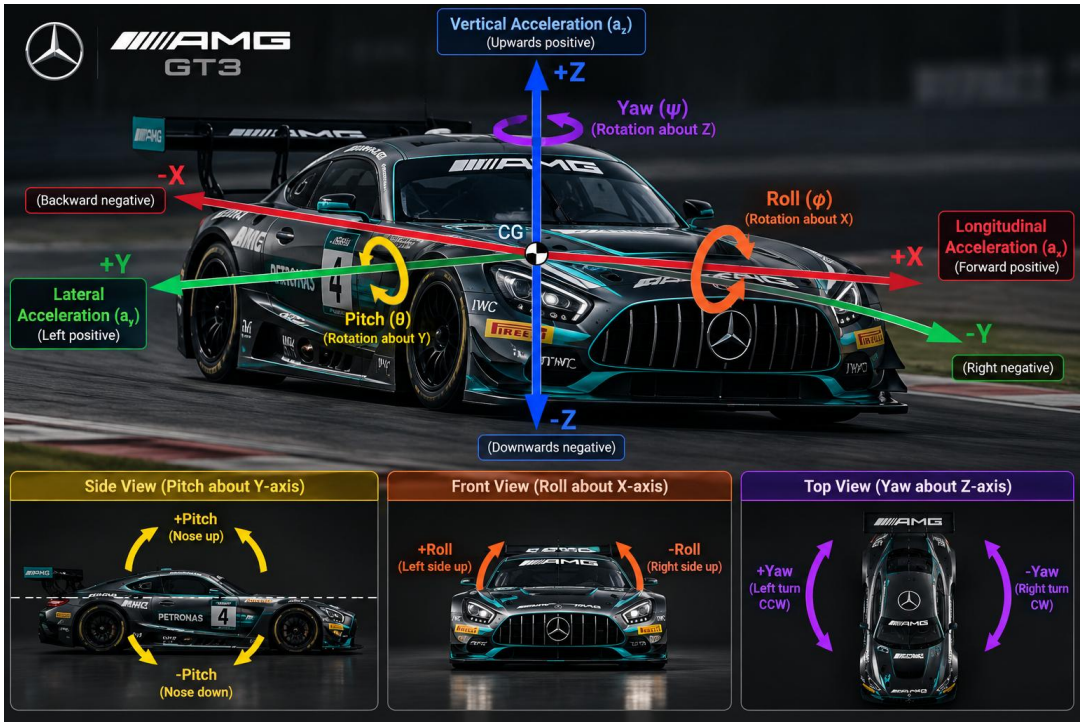


Figure 1.1: Vehicle Axis System and the Six Measured Motions

Modern cars depend on this information. Electronic stability control compares the measured yaw rate with the yaw rate the driver is asking for through the steering wheel and brakes individual wheels when the two do not match. Airbag and rollover systems watch for sudden spikes in acceleration and roll rate. Fleet telematics boxes use the same signals to flag harsh braking and aggressive cornering.

The sensors behind these systems are now remarkably cheap. A MEMS IMU module such as the MPU6050 combines a three-axis accelerometer and a three-axis gyroscope on a board the size of a thumbnail. The difficulty is no longer getting a sensor; it is getting **trustworthy data**

out of it. The sensor has to be fixed firmly, sampled at a steady rate, stored reliably and, most importantly, read with an understanding of what it can and cannot tell you.

This project explores that whole chain on a small scale. We designed and built a portable six-axis logger, mounted it on a model car, recorded a test run and then worked through the data channel by channel. Along the way the data showed us not only what the car was doing, but also a couple of things about our own setup that we would not have noticed without looking closely.

2. Project Objectives

The main objective was to build a low-cost system that records how a vehicle moves and to show that its data can be interpreted in a meaningful way. Rather than stopping at a working circuit, the project was meant to carry the work all the way through to analysis and a critical look at the results.

The specific objectives were:

- To measure **longitudinal, lateral and vertical acceleration** of a vehicle.
- To measure **roll, pitch and yaw angular velocity**.
- To acquire the sensor data with a **microcontroller-based system** at a steady sampling rate.
- To **store the data on board** in a format that can be opened on any computer.
- To design a **rigid, low-cost mounting** that keeps the sensor aligned with the vehicle axes.
- To **process and visualise** the data using MATLAB.
- To **identify and interpret** acceleration, braking, cornering, vibration and impact events in the recorded run.
- To **validate the system** on a small-scale vehicle platform and identify its limitations.

Taken together, these objectives cover the full path from a physical movement of the car to an engineering explanation of that movement.

3. System Design

3.1 System Architecture

The system is deliberately simple. Motion of the vehicle is sensed by the IMU, read by the ESP32, written to the microSD card and analysed later in MATLAB (Figure 3.1). Keeping storage on board meant there was no wireless link to drop out and no cable to trail behind the car.

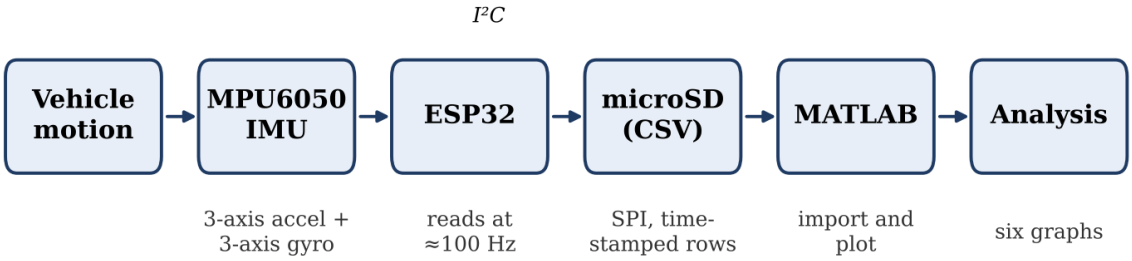


Figure 3.1: System Architecture of the IMU Logger

- **Sensing:** the MPU6050 measures 3-axis acceleration and 3-axis angular velocity.
- **Data acquisition:** the ESP32 reads the sensor over the I²C bus about every 10 ms.
- **Data logging:** each reading is time-stamped and saved as a CSV row on the microSD card.
- **Data analysis:** the CSV file is imported into MATLAB, plotted and interpreted.

3.2 Hardware Components

All the parts used are inexpensive, widely available modules (Figure 3.2 and Table 3.1).

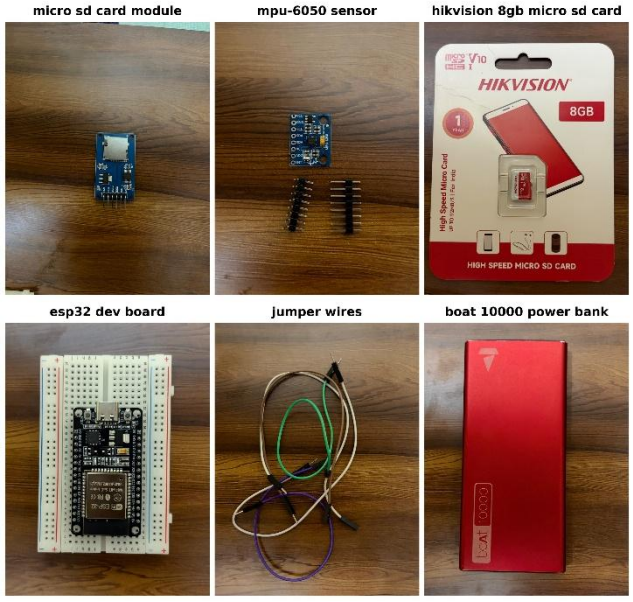


Figure 3.2: Hardware Components Used in the Logger

Component	Role in the system
MPU6050 IMU	3-axis accelerometer and 3-axis gyroscope with 16-bit outputs, read over I²C
ESP32-WROOM-32	Main controller: reads, converts, time-stamps and stores each sample
microSD SPI module	Connects the SD card to the ESP32
HIKVISION 8 GB microSD card	Stores the CSV data file
boAt 10,000 mAh power bank	Portable 5 V supply carried on the car
Breadboard and jumper wires	Circuit development and bench testing
RC model car	Small-scale test platform

Table 3.1: Hardware Components and Their Roles

The MPU6050 offers several full-scale ranges. On its most sensitive settings, ± 2 g and ± 250 °/s, one g corresponds to 16,384 counts and one degree per second to 131 counts. A sensitive range gives fine resolution but runs out of headroom sooner, a trade-off that turned out to matter in this test (Section 7.3).

3.3 Wiring

The IMU uses the ESP32's default I²C pins and the SD module uses the VSPI bus (Figure 3.3, Table 3.2). The IMU's interrupt pin is left unconnected because the ESP32 reads it on a fixed 10 ms timer.

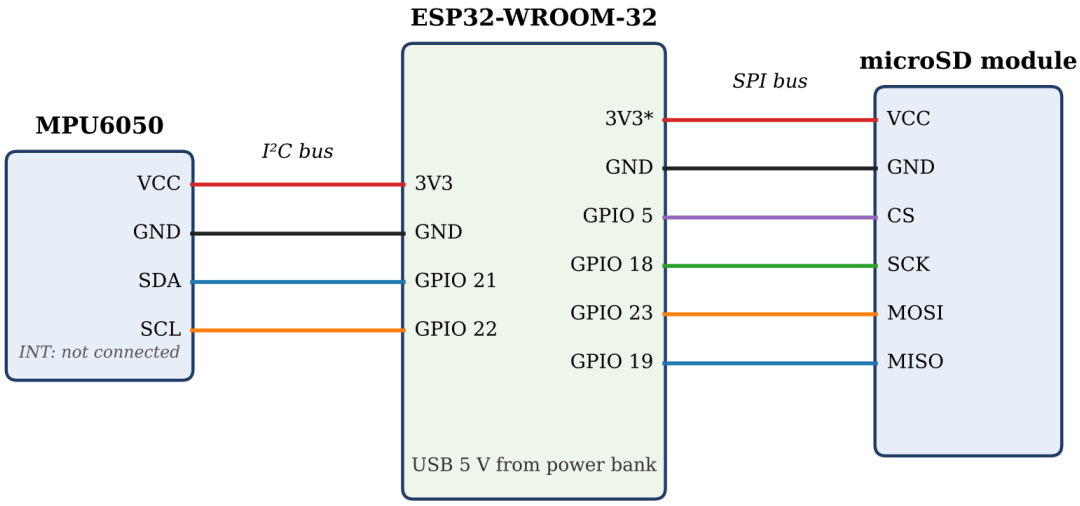


Figure 3.3: Wiring of the MPU6050 and microSD Module to the ESP32

Module pin	ESP32 pin	Function
MPU6050 VCC / GND	3.3 V / GND	Power
MPU6050 SDA	GPIO 21	I²C data
MPU6050 SCL	GPIO 22	I²C clock
MPU6050 INT	Not connected	–
microSD VCC / GND	3.3 V / GND	Power
microSD CS	GPIO 5	SPI chip select
microSD SCK	GPIO 18	SPI clock
microSD MOSI	GPIO 23	Data from ESP32 to card
microSD MISO	GPIO 19	Data from card to ESP32

Table 3.2: Pin Connections

3.4 3D-Printed Sensor Holder

A loose sensor is worse than no sensor at all. If the board can rattle, every rattle shows up as false vibration, and if it can shift, its axes no longer line up with the car's. To avoid this, a three-part holder was designed and 3D printed (Figure 3.4):

- **Enclosure frame:** an open rectangular box that surrounds the electronics and holds them in one place.
- **Cover plate:** a flat plate with a window for cable access and for seeing the boards.
- **Support block:** a slotted block that grips the modules so nothing moves during a run.



Figure 3.4: The Three Printed Parts of the Sensor Holder

3.5 Final Prototype on the RC Car

The finished holder sits on the roof of the model car, strapped down with cable ties, with the power bank on top (Figure 3.5). The intention was for the sensor's X-axis to point forward, Y sideways and Z up. As Section 7.4 shows, the data later suggested the board had ended up facing the other way.

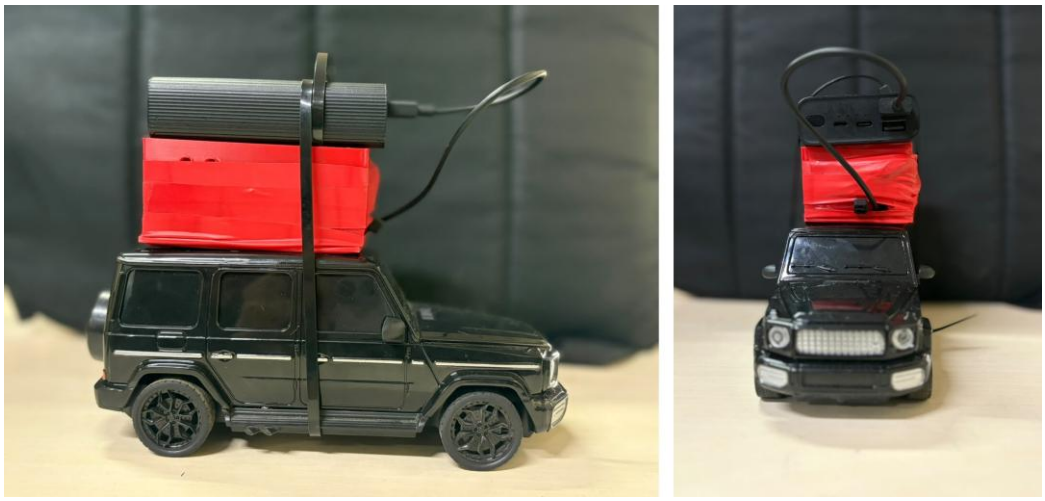


Figure 3.5: Final Prototype – Side View (left) and Front View (right)

4. Data Acquisition and Logging

4.1 Sampling and File Format

Every 10 ms the ESP32 reads the six sensor values, converts them into g and degrees per second, adds the time from its millisecond timer and writes one line to the card. The result is a plain CSV file that opens directly in MATLAB, Excel or Python. The recorded run lasted about **17.8 s** at roughly **100 Hz**, giving around **1,780 samples** per channel.

Parameter	**Value**
Run duration	≈ 17.8 s
Sample interval	10 ms
Sampling rate	≈ 100 Hz
Samples per channel	≈ 1,780
Channels	6
File	RUN007.csv

Table 4.1: Summary of the Recorded Run

4.2 Sample Raw Data

Before any plotting, the first rows of the file were inspected to confirm that the logging had worked (Figure 4.1). Each row is one moment in time and each column one channel. The timestamps step by 10 ms, apart from the very first interval of 9 ms, which confirms a steady sampling rate.

Time_ms	Ax_g	Ay_g	Az_g	Gx_dps	Gy_dps	Gz_dps
2	0.0552	0.069	0.9912	1.064	0.484	-0.463
11	0.0601	0.0737	0.9907	0.72	0.682	-0.265
21	0.054	0.0727	0.9924	1.216	0.56	-0.562
31	0.0626	0.0681	0.9887	1.194	0.843	-0.723
41	0.0643	0.069	0.9863	1.216	0.553	-0.547
51	0.0628	0.0729	0.9892	1.049	0.705	-0.456
61	0.0621	0.0717	0.9919	0.987	0.881	-0.395
71	0.0574	0.0673	0.9948	1.27	0.576	-0.356
81	0.0623	0.0732	0.9921	1.323	0.667	-0.746
91	0.064	0.0717	0.9909	0.652	0.69	-0.616
101	0.0584	0.0734	0.9912	1.133	0.843	-0.555
111	0.0592	0.0715	0.9882	1.087	0.713	-0.303
121	0.0594	0.0707	0.9938	0.881	0.957	-0.265
131	0.0611	0.0698	0.9909	0.957	0.66	-0.311
141	0.0638	0.0715	0.9863	0.865	0.4	-0.494
151	0.0618	0.0732	0.9921	1.224	0.591	-0.257
161	0.057	0.0698	0.9946	1.079	0.24	-0.372
171	0.0599	0.0729	0.9992	1.262	0.652	-0.303
181	0.0577	0.072	0.9931	1.178	0.354	-0.356
191	0.0623	0.0705	0.986	1.178	0.484	-0.44
201	0.0604	0.0712	0.9963	0.949	0.713	-0.692
211	0.056	0.0722	0.9887	0.789	0.705	-0.211
221	0.0592	0.0688	0.9963	0.987	0.782	-0.295
231	0.057	0.0727	0.9863	1.01	0.247	-0.517

Figure 4.1: First Rows of the Logged CSV File

4.3 Available Data Channels

Channel	Quantity	Unit	What it describes
Time_ms	Timestamp	ms	Time since the ESP32 started
Ax_g	Longitudinal acceleration	g	Acceleration and braking
Ay_g	Lateral acceleration	g	Side load in corners
Az_g	Vertical acceleration	g	Gravity plus bumps and vibration
Gx_dps	Roll rate	°/s	Rotation about the X-axis
Gy_dps	Pitch rate	°/s	Rotation about the Y-axis
Gz_dps	Yaw rate	°/s	Rotation about the Z-axis (turning)

Table 4.2: Description of the Logged Channels

4.4 Static Baseline Check

The first 23 rows were recorded with the car standing still, which makes them a free calibration check. An accelerometer at rest measures gravity, so a level sensor should read 0 g on X and Y and 1 g on Z, and a still gyroscope should read zero.

Channel	Mean	Std. deviation	Ideal
Ax	0.060 g	0.003 g	0 g
Ay	0.071 g	0.002 g	0 g
Az	0.992 g	0.003 g	1 g
Gx	+1.05 °/s	0.18 °/s	0 °/s
Gy	+0.64 °/s	0.17 °/s	0 °/s
Gz	−0.43 °/s	0.16 °/s	0 °/s

Table 4.3: Sensor Readings at Rest (First 23 Samples)

The total measured acceleration works out to:

$$|a| = \sqrt{(0.060^2 + 0.071^2 + 0.992^2)} = 0.996 \text{ g}$$

That is within 0.4 % of 1 g, so the accelerometer scale is sound. The small X and Y readings would correspond to a tilt of about **3.5° and 4°**, which is believable for a holder strapped to a toy car's roof, though some of it may be the sensor's own offset. The gyroscope offsets of up to about 1 °/s are typical for an uncalibrated MEMS part. They look tiny, but they add up: integrating a yaw offset of 0.43 °/s over this run would put the estimated heading about 7.7° out by the end.

5. Acceleration Analysis

The plots in this chapter and the next show the whole run. Two features appear in every one of them and are worth pointing out first. The car was stationary from **about 6.5 to 9.5 s** and again, briefly, from **about 13.4 to 14.1 s**. During these periods every trace goes flat, which makes them a useful reference when reading the rest of the graphs.

5.1 Longitudinal Acceleration

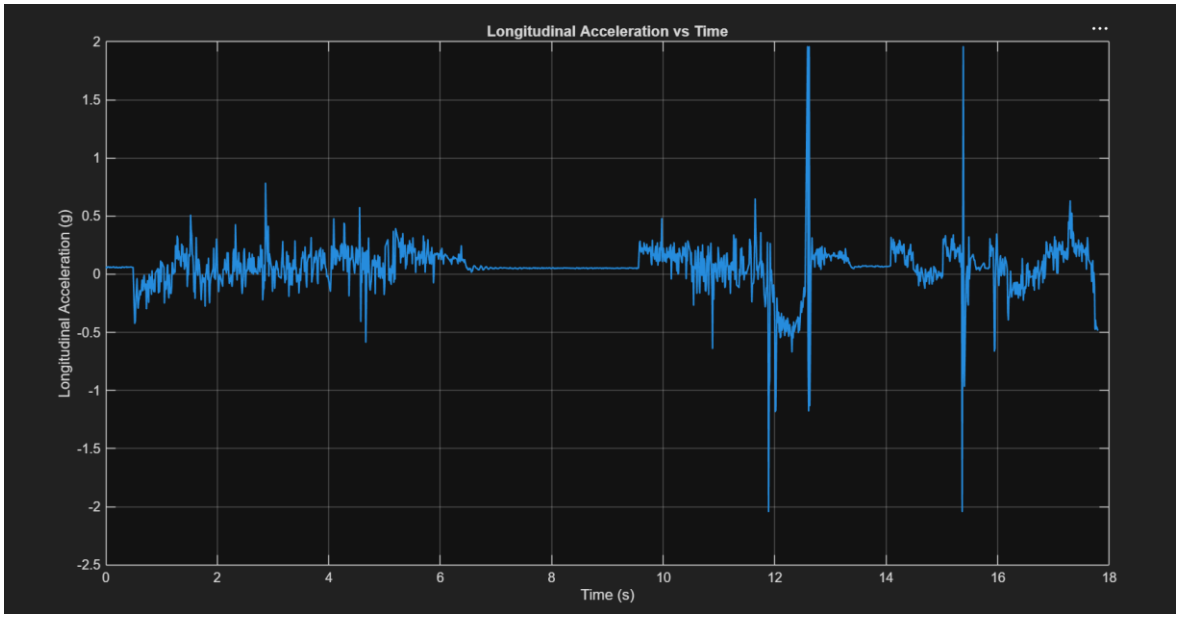


Figure 5.1: Longitudinal Acceleration vs Time

The longitudinal signal sits at its 0.06 g offset until the car moves off at about 0.5 s, where it drops sharply to around −0.4 g. During normal running it stays mostly between −0.3 g and +0.5 g, with short peaks up to about 0.8 g.

The most interesting stretch is between **12.0 and 12.5 s**, where the signal holds at about **−0.45 g for half a second** while the yaw rate is almost zero. A sustained level like this, with no turning, is what a straight-line acceleration or braking phase looks like. Either side of it are sharp spikes to about −2.05 g and +1.95 g, and another pair of about ± 2 g appears at 15.4 s. These are discussed in Chapter 7.

5.2 Lateral Acceleration

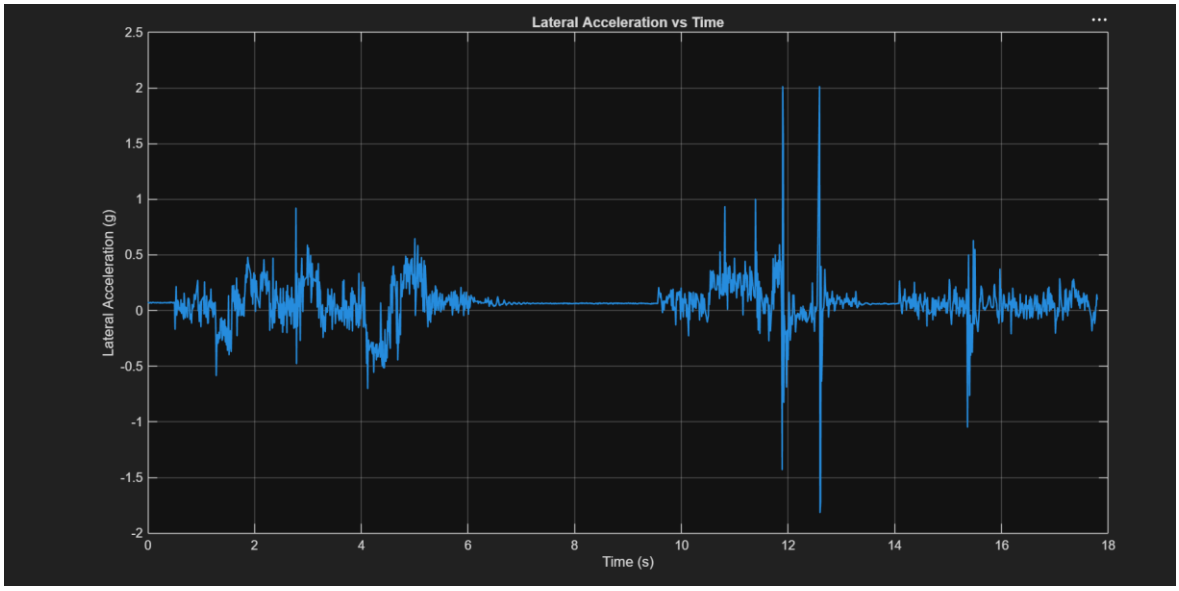


Figure 5.2: Lateral Acceleration vs Time

Lateral acceleration is the side load the car feels in a corner. For most of the run it stays within **± 0.5 g**, and there are clear blocks of sustained side load, for example about −0.4 g between 4.1 and 4.6 s and about +0.35 g between 4.8 and 5.3 s. These blocks line up exactly with turns in the yaw-rate plot, which is a good sign that the two sensors agree.

The tall, needle-like spikes near 11.9 s and 12.6 s, reaching about +2 g and −1.8 g, are a different kind of event. They last only a few samples, far too short to be a corner.

5.3 Vertical Acceleration

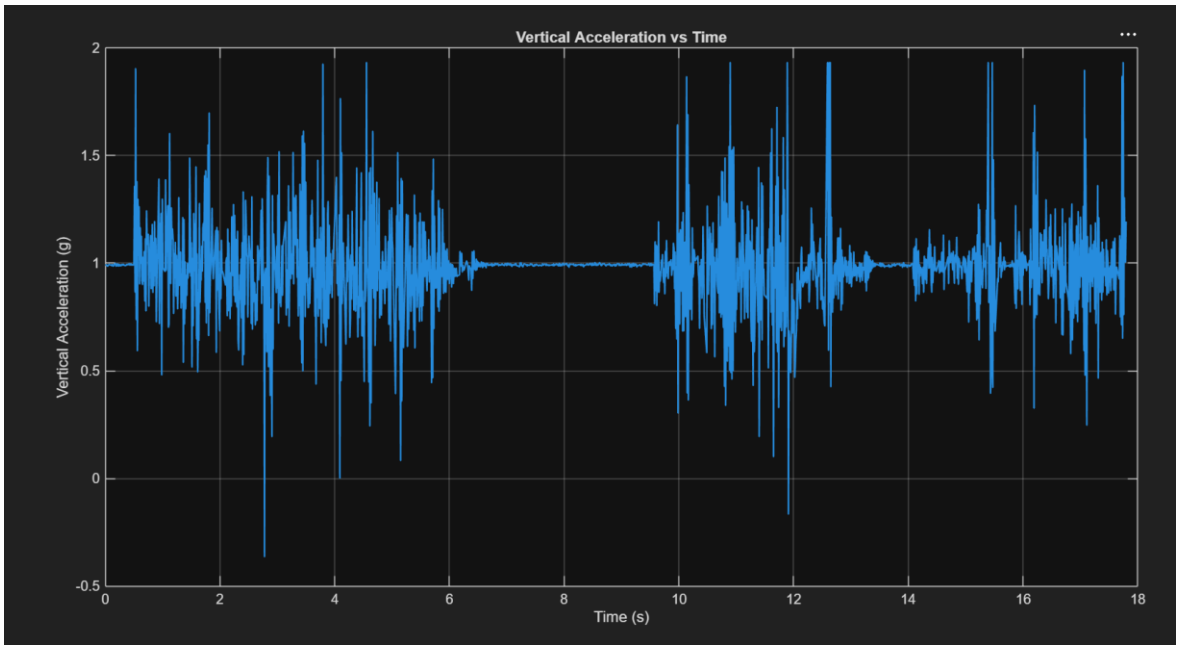


Figure 5.3: Vertical Acceleration vs Time

The vertical channel always includes gravity, so it centres on **1 g**. Whenever the car is moving the trace becomes very busy, with frequent peaks near 1.9 g and dips to 0.3 – 0.5 g; the lowest value, about −0.36 g, occurs at 2.8 s. A value close to zero means the sensor is momentarily close to free fall, as when a wheel drops off a small bump.

The amount of activity says more about the car than the road. A model car has very stiff suspension and almost no isolation, so road vibration passes straight up to the roof. A full-size car with proper springs and dampers would show a much calmer trace from the same sensor.

6. Angular Velocity Analysis

6.1 Roll Rate

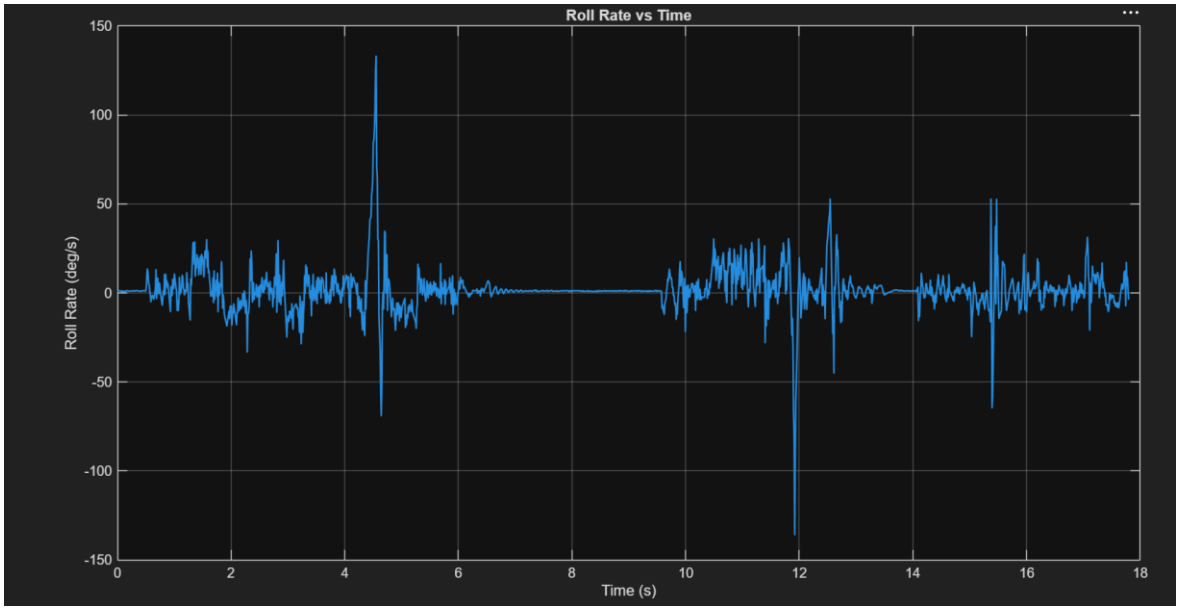


Figure 6.1: Roll Rate vs Time

Roll rate measures how quickly the body leans from side to side. It stays mostly within **± 30 °/s**, with isolated swings of about +133 °/s at 4.6 s, −137 °/s at 11.9 s and around ± 55 °/s at 12.6 s and 15.4 s. The small-amplitude activity follows the moving periods of the vertical channel closely, which suggests that most of the roll motion comes from the body rocking over uneven ground rather than leaning steadily in corners.

6.2 Pitch Rate

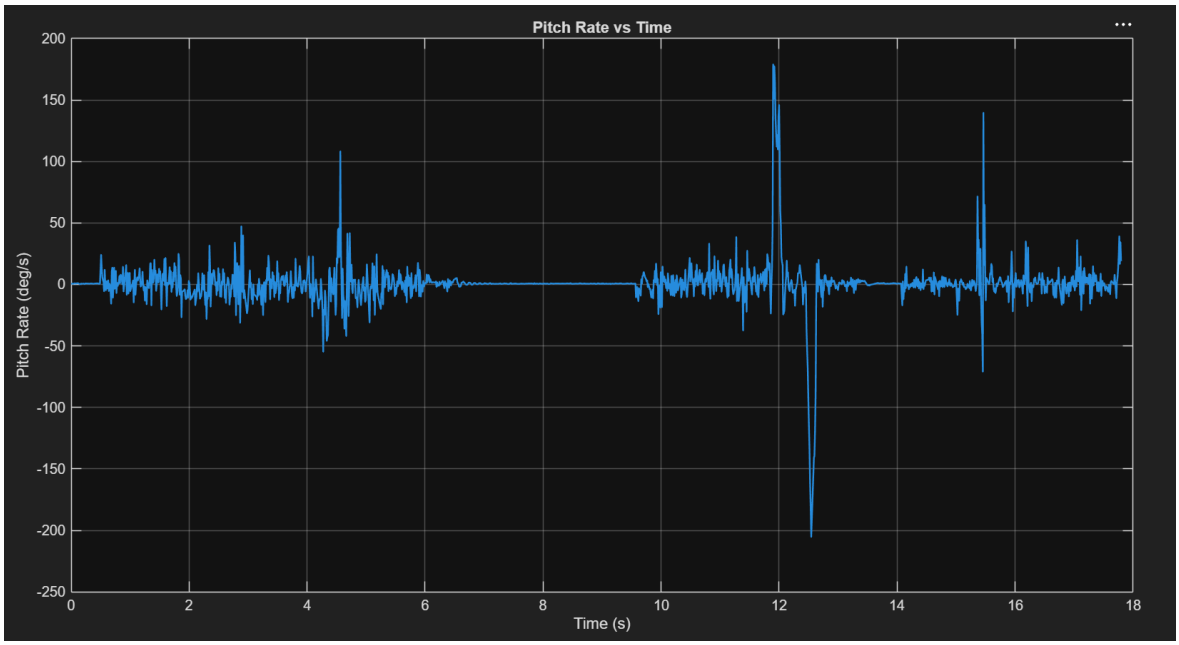


Figure 6.2: Pitch Rate vs Time

Pitch rate describes the nose moving up and down. Again, most of the run sits within ± 30 °/s. The standout feature is a swing to about **+178 °/s at 11.9 s** followed by **−205 °/s at 12.55 s**, with a smaller pair of +140 and −72 °/s at 15.4 s. A rapid pitch one way and then straight back is what you would expect if the car hit something or landed hard, not from smooth acceleration or braking.

6.3 Yaw Rate

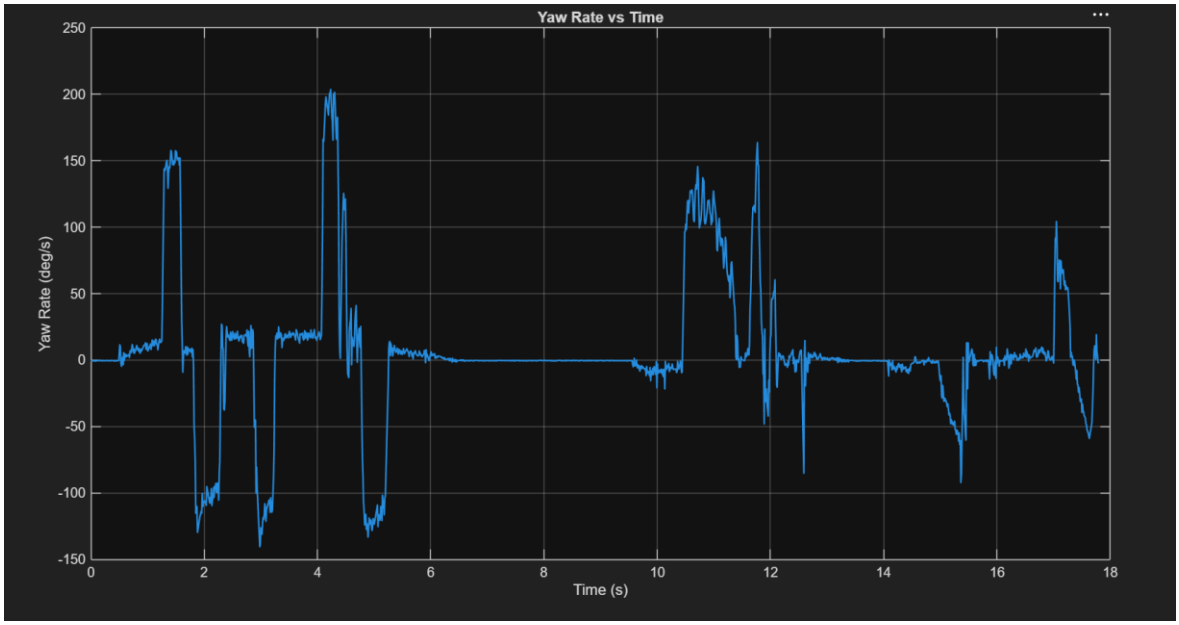


Figure 6.3: Yaw Rate vs Time

If one graph tells the story of the run, it is this one. Instead of continuous vibration, the yaw rate shows **clean plateaus**, and each plateau is a turn. In the first six seconds the car made a series of alternating turns at roughly +150, −110, −130, +200 and −120 °/s. After the long stop it made a longer turn at +100 to +140 °/s between 10.5 and 11.4 s, a gradual turn reaching about −90 °/s around 15.4 s and a final turn near +100 °/s at 17 s.

Because yaw rate is the speed of turning, the area under each plateau gives the change in heading:

$$\Delta\psi = \int r \, dt$$

The 200 °/s plateau near 4.3 s lasts about 0.4 s, which works out to a heading change of roughly **80°**. The 150 °/s plateau near 1.5 s, lasting about 0.3 s, gives about **45°**. With the gyroscope offset removed, the same idea could be used to rebuild the path the car took.

7. Reading the Channels Together

Looking at each channel on its own only goes so far. Some of the most useful conclusions came from comparing them.

7.1 Turns: Yaw Rate and Lateral Acceleration

In a steady corner a car turns (yaw rate) and feels a side load (lateral acceleration) at the same time, so the two should rise and fall together. In this run they do: every yaw plateau has a matching block of lateral acceleration of 0.3 – 0.5 g. This cross-check gives confidence that the turning events are real and not artefacts of one sensor.

7.2 Impact Events

At about **11.9 s, 12.6 s and 15.4 s**, all six channels spike within a few samples of each other. A normal manoeuvre mainly affects one or two related channels. A short, simultaneous spike on all of them is the fingerprint of an **impact**, such as the car hitting an obstacle or landing after leaving the ground. This is exactly the type of signal that crash detection and airbag systems are designed to pick out.

Time	Longitudinal	Lateral	Roll rate	Pitch rate
≈ 11.9 s	−2.05 g	+2.0 / −1.4 g	−137 °/s	+178 °/s
≈ 12.6 s	+1.95 g	+2.0 / −1.8 g	± 50 °/s	−205 °/s
≈ 15.4 s	± 2 g	+0.6 / −1.05 g	+53 / −65 °/s	+140 / −72 °/s

Table 7.1: Peak Values During the Impact Events

7.3 Sensor Range Check

Looking closely at the impact spikes, their tops are **flat at about ± 2 g** on the longitudinal and lateral channels, and many vertical peaks flatten at about 1.93 g. When several axes all stop at the same value, the sensor has usually run out of range rather than the motion happening to stop there. This tells us the accelerometer was on its ± 2 g setting and that the real impact accelerations were higher than recorded. For future tests a ± 8 g range would give enough headroom.

7.4 Axis Orientation Check

This was the finding we least expected. In the vehicle axis system, a left turn gives a **positive** yaw rate and a **positive** lateral acceleration, pointing to the left. In our data the two have **opposite signs in every turn**, for example +200 °/s of yaw with about −0.4 g of lateral acceleration. The longitudinal channel also goes negative, not positive, when the car first moves off.

Both observations fit one explanation: the sensor board was mounted **turned through 180°** about its vertical axis, so its X-axis pointed backward and its Y-axis to the right. The sizes and timings of all the events are unaffected, but the signs of the longitudinal and lateral channels

have to be read the other way round. The simple fix is to tilt the car by hand before every run and check which way each reading moves.

8. Automotive Applications

The six channels recorded here are the same ones that several production vehicle systems rely on. Production cars use automotive-grade IMUs with much tighter specifications, but the physics, and the way the data is read, is the same.

Application	How the IMU data is used
Electronic stability control	Compares measured yaw rate and lateral acceleration with the driver's intended path and brakes individual wheels to correct it.
Rollover detection	Combines roll rate with lateral and vertical acceleration to predict a rollover and fire curtain airbags.
Crash and airbag detection	Looks for sudden multi-axis spikes like those at 11.9 s and 15.4 s, and feeds event data recorders.
Driver behaviour and telematics	Flags harsh braking, hard acceleration and sharp cornering for fleet monitoring and insurance.
Ride and durability testing	Uses vertical acceleration and pitch and roll rates to assess suspension and vibration.
Navigation and hill-hold	Bridges gaps in GPS coverage and uses gravity components to measure road gradient.

Table 8.1: How IMU Data Is Used in Vehicles

9. Limitations and Future Work

Being clear about what the system cannot yet do is as important as showing what it can. The main limitations of this first version are:

- **No calibration.** The offsets in Section 4.4 remain in the data and would cause drift if the signals were integrated.
- **Range too small for impacts.** The ± 2 g setting clipped the largest peaks.
- **Orientation not verified.** The board appears to have been mounted reversed, which was only discovered during analysis.
- **No filtering.** Vibration dominates some channels, and at 100 Hz any content above 50 Hz can alias into the data.
- **One run on one platform.** The results have not been compared against a reference instrument or repeated on a full-size vehicle.

Each limitation points to a clear next step:

- Calibrate the sensor in several known orientations and subtract the offsets.
- Use the ± 8 g accelerometer range and check orientation with a tilt test before each run.
- Add low-pass filtering and a higher sampling rate.
- Add **GPS** so that acceleration can be linked to speed and position.
- Apply **sensor fusion** (a complementary or Kalman filter) to estimate roll, pitch and heading.
- Stream live data over the ESP32's built-in Wi-Fi or Bluetooth.
- Test the system on a **full-size vehicle**.

10. Conclusions

The goal of this project was to find out how much real vehicle behaviour a low-cost IMU logger can capture, and to read that data critically rather than simply plotting it.

A compact logger was built from an MPU6050, an ESP32 and a microSD module, held in a 3D-printed mount and powered by a power bank. It recorded six motion channels at about 100 Hz for a 17.8 s run on a model car without losing any data. The static check confirmed that the sensor was working and close to level, with a total reading of 0.996 g and low noise.

The moving data clearly separated the different things the car did. **Yaw rate** showed each turn as a clean plateau of 100 – 200 °/s, matched by **lateral acceleration** of 0.3 – 0.5 g. **Two stops** appeared as flat lines on every channel. A sustained **longitudinal** level of about −0.45 g marked a straight-line acceleration or braking phase. The **vertical** channel showed how directly road vibration reaches the body of a small car, and simultaneous spikes on all channels identified **three impact events**.

Just as valuable were the lessons about the measurement itself. The flat-topped peaks showed that the accelerometer range was too small, and comparing the signs of yaw rate and lateral acceleration revealed that the board had been mounted the wrong way round. Neither problem was visible during the build; both were found only by reading the data carefully.

Overall, the project showed that a sensor costing very little can produce genuinely useful vehicle dynamics data, provided it is mounted rigidly, configured sensibly and interpreted with care. With calibration, filtering and a full-scale test, the same system could become a practical tool for vehicle testing.

11. References

[1] T. D. Gillespie, Fundamentals of Vehicle Dynamics, SAE International, Warrendale, Pennsylvania, USA, 1992.

[2] R. Rajamani, Vehicle Dynamics and Control, 2nd Edition, Springer, New York, USA, 2012.

[3] InvenSense Inc., MPU-6000 and MPU-6050 Product Specification, Revision 3.4, Sunnyvale, California, USA, 2013.

[4] Espressif Systems, ESP32 Technical Reference Manual, Espressif Systems, Shanghai, China.

[5] O. J. Woodman, An Introduction to Inertial Navigation, Technical Report UCAM-CL-TR-696, University of Cambridge Computer Laboratory, 2007.

[6] A. T. van Zanten, “Bosch ESP Systems: 5 Years of Experience,” SAE Technical Paper 2000-01-1633, SAE International, 2000.

[7] The MathWorks Inc., MATLAB Documentation, Natick, Massachusetts, USA.

Appendix A – MATLAB Script

This appendix contains the MATLAB script used to import the logged CSV file, convert the timestamps to seconds and plot each of the six motion channels against time.

```
clc;
clear;
close all;
% Load telemetry data
data = readtable('RUN007.csv');
% Convert milliseconds to seconds
t = (data.Time_ms - data.Time_ms(1)) / 1000;
%% ACCELERATION
% 1. Longitudinal Acceleration
figure;
plot(t, data.Ax_g, 'LineWidth', 1.2);
grid on;
title('Longitudinal Acceleration vs Time');
xlabel('Time (s)');
ylabel('Longitudinal Acceleration (g)');
% 2. Lateral Acceleration
figure;
plot(t, data.Ay_g, 'LineWidth', 1.2);
grid on;
title('Lateral Acceleration vs Time');
xlabel('Time (s)');
ylabel('Lateral Acceleration (g)');
% 3. Vertical Acceleration
figure;
plot(t, data.Az_g, 'LineWidth', 1.2);
grid on;
title('Vertical Acceleration vs Time');
xlabel('Time (s)');
ylabel('Vertical Acceleration (g)');
%% ROTATIONAL VELOCITY
% 4. Roll Rate (Rotation about X-axis)
figure;
plot(t, data.Gx_dps, 'LineWidth', 1.2);
grid on;
title('Roll Rate vs Time');
xlabel('Time (s)');
ylabel('Roll Rate (deg/s)');
% 5. Pitch Rate (Rotation about Y-axis)
figure;
plot(t, data.Gy_dps, 'LineWidth', 1.2);
grid on;
title('Pitch Rate vs Time');
xlabel('Time (s)');
ylabel('Pitch Rate (deg/s)');
% 6. Yaw Rate (Rotation about Z-axis)
figure;
plot(t, data.Gz_dps, 'LineWidth', 1.2);
grid on;
title('Yaw Rate vs Time');
xlabel('Time (s)');
ylabel('Yaw Rate (deg/s)');
```